

Prior MoE Approaches for Imbalanced Network Intrusion Detection

A Review of Attempts Before the code_1/2/3 Series

Research Summary

Network Security & Class Imbalance Project

April 23, 2026

- 1 Problem Statement
- 2 Overview of Attempts
- 3 Attempt 1: TAILGUARD — `cic_6.py`
- 4 Attempt 2: XGBoost Router + SMOTE — `cic_xg_19.py`
- 5 Attempt 3: OOD Competence Ensemble — `code_final.py`
- 6 Attempt 4: Temporal Splits — `code_chrono` & `code_file`
- 7 Attempt 5: FAR-MoE — `code_mat.py`
- 8 Failure Analysis
- 9 Summary & Open Questions

Problem: Severely Imbalanced Multi-Class IDS

Datasets

- CIC-IDS2017 / CIC-IDS2018
- UNSW-NB15 / NF-UNSW-NB15

Core challenge

- Imbalance ratio (IR) $> 1,000$ for tail attack classes
- *Heartbleed*: 9 train samples in CIC-IDS2017
- *Infiltration*: 22 train samples
- XGBoost baseline already learns benign / head classes well — tail-class recall collapses

Goal

- Beat single XGBoost baseline on **tail-class F1**
- Without sacrificing majority-class precision

CIC-IDS2017 Tail Classes (val)

Class	Train n	Baseline Recall
Bot	1,180	0.812
Heartbleed	7	0.500
Infiltration	22	0.857
Web-BF	905	0.847
Web-SQLi	13	0.250
Web-XSS	391	0.275

Timeline of All Prior Attempts

Script	Core Idea	Split	Outcome
cic_6.py	Taxonomy-aware MoE (TAILGUARD)	Random	No gain
cic_xg_19.py	XGBoost router + SMOTE per expert	Random	No gain
code_final.py	OOD-filtered competence ensemble	Random	No gain
code_final_nf.py	Same on NF-UNSW-NB15	Random	Cross-dataset ref.
code_chrono.py	Chronological split (anti-leakage)	Chrono	Degraded
code_file.py	Per-file split (anti-leakage)	Per-file	Degraded
code_mat.py	FAR-MoE: CM-derived experts + OOD gating	Chrono	Val ≈ 1 , test no gain

Persistent finding

In every attempt, the single XGBoost baseline equals or outperforms the MoE on the test set.

Architecture

- Explicitly maps attack classes to 4 taxonomy groups (volumetric, slow/low, web, reconnaissance)
- One XGBoost expert per group + 5 random specialists
- Meta-classifier (Logistic Regression) selects which experts to query at inference time
- Competence score = $P(\text{expert correct} \mid x)$

Training

- Random stratified 60/20/20 split
- Meta-classifier trained on val predictions

Results (CIC-IDS2017)

Metric	Baseline	MoE
Accuracy	0.9989	0.9989
Macro F1	0.8380	0.8463
Weighted F1	0.9989	0.9989

Why it failed

Router itself misclassifies tail-class samples (rare $\Rightarrow$ hard to learn), so those samples are routed to the wrong expert and perform *worse* than baseline.

Key changes from TAILGUARD

- Router is also an XGBoost (`multi:softprob`), using both original features + per-expert uncertainty
- SMOTE applied *per expert* to oversample minority classes within each expert's training set
- Configurable: `-assignment_mode type|random`, `-router_target expert`
- Soft routing: each expert weighted by router probability

Hypothesis

- A more powerful router would fix the misrouting problem
- SMOTE would alleviate expert-level imbalance

Root problem exposed

XGBoost's boosting *already* implements implicit MoE — it iteratively reweights hard (minority) samples. Adding an explicit router on top offers no additional specialization signal.

Additional failure mode

SMOTE within each expert creates synthetic samples that may not lie on the true data manifold, and the oversampled expert still sees fewer real tail samples than the global model.

Design shift: abandon explicit routing

- Each expert always sees all training data
- At inference: compute per-expert *OOD score* and *competence score*
- OOD score: Mahalanobis distance on expert's training distribution (LedoitWolf covariance)
- Competence: LR trained on val to predict “was this expert correct on this sample?”
- Experts with high OOD z-score are excluded; remaining experts are weighted by competence

Results (UNSW-NB15)

Metric	Baseline	MoE
Accuracy	0.8335	0.8145
Macro F1	0.5788	0.6065
Weighted F1	0.8137	0.8215

Observation

Macro F1 improves slightly but **accuracy drops**. Competence model trained on val does not generalise to test distribution — especially under random splits where val and test are i.i.d. but experts overfit to val.

Motivation

- Random stratified splits allow future data to “leak” into training via shared temporal context (same capture session)
- Real-world IDS must generalise to *unseen future attacks*

Two strategies

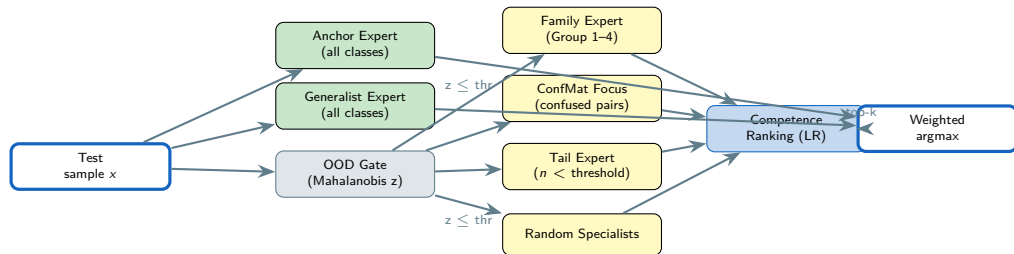
- 1 **Chrono**: within each class, sort by time; first 60% train, next 20% val, last 20% test
- 2 **Per-file**: assign whole capture files to train/val/test; no sample from the same file in two splits

Both strategies degraded performance

- Tail classes have very few samples; chronological ordering leaves test with the “hardest” temporal tail
- Distribution shift between train and test is large under file-wise split
- MoE experts specialised on early-time patterns fail on later-time attacks even more than the baseline

Lesson

Preventing leakage is necessary but makes the imbalance problem *strictly harder* — the two problems must be solved jointly.



Feature views per expert: volume (byte/packet counts), timing (IAT, duration), packet (flag/size), tcp, all.

ConfMat Focus:

confusion pairs derived from a baseline probe on the *validation* split.

Selected per-class results (CIC-IDS2017 test)

Class	Recall(B)	Recall(MoE)	
Web-XSS	0.231	0.838	Some tail
Bot	0.756	0.804	
Web-BF	0.850	0.502	
Web-SQLi	0.000	0.000	
Infiltration	0.571	0.571	
Macro F1	0.838	≈same	

classes improve, others regress — net effect is neutral.

Critical failure: CM leakage

- The ConfMat Focus expert is designed using the *confusion matrix on the validation split of the same dataset*
- Confusion patterns on val $\neq$ confusion patterns on test, especially under chronological splits
- Val accuracy looks near-perfect (probe sees similar distribution as its own training set)
- At test time, the "focus expert" targets the wrong class pairs

#	Failure Mode	Observed in
1	Routing error compounds imbalance. Tail classes are rare $\Rightarrow$ router cannot learn them $\Rightarrow$ tail samples routed to wrong expert $\Rightarrow$ worse than baseline.	cic_6, cic_xg_19
2	Expert-level imbalance unresolved. Partitioning classes across experts does not change the within-expert IR; each specialist still sees too few tail samples to specialise.	All MoE variants
3	CM leakage / val overfitting. Confusion-matrix-derived expert design exploits val distribution; patterns do not transfer to test (especially chrono/file splits).	code_mat
4	XGBoost implicit MoE. Gradient boosting already reweights hard examples iteratively. Explicit routing adds overhead but no new specialisation signal.	All MoE variants

Negative results (confirmed)

- Taxonomy-based routing $\Rightarrow$ no gain
- XGBoost router + SMOTE $\Rightarrow$ no gain
- OOD + competence weighting $\Rightarrow$ no gain
- Confusion-matrix expert design $\Rightarrow$ val leakage
- Temporal splits make imbalance harder, not easier

Open research questions

- 1 **Unbiased expert design:** how to partition classes into experts such that each expert genuinely sees different feature-space structure, not just different class labels?
- 2 **Routing under imbalance:** can a router be trained without seeing tail classes, yet still route them correctly?
- 3 **Leakage-free temporal evaluation:** how to design an ensemble that improves *and* degrades gracefully under distribution shift?
- 4 **Beyond XGBoost:** does the implicit-MoE argument hold for other base learners (LightGBM, CatBoost, neural nets)?

Next: code_1/2/3 series

Address the unbiased expert design problem directly by ... (see subsequent slides)

Appendix: Experiment Settings

Setting	Value
Base learner	XGBoost (multi:softprob)
Split (random)	Stratified 60 / 20 / 20
Split (chrono)	Episode-based attack, chronological normal 60/20/20
Split (per-file)	Whole capture files assigned to train/val/test
SMOTE threshold	1,000 samples per class
OOD model	LedoitWolf covariance $\Rightarrow$ Mahalanobis distance
Competence model	Logistic Regression on val
Random seed	42
Datasets	CIC-IDS2017, UNSW-NB15, NF-UNSW-NB15